

사주(Saju) 서비스 기술 설계 문서

작성일: 2026-06-24

버전: v2.0

대상: 개발자 · 기획자 공통

범위: 백엔드(Spring Boot) · 인프라(AWS)

이 문서는 사주(Saju) 서비스 개발 과정에서 내린 주요 기술 결정을 기록한다.

각 결정마다 배경 / 결정 / 근거 / 결과 / 대안 / 향후 확장 가능성을 명시해 '왜 이렇게 만들었는가'를 미래의 팀원과 자신에게 설명한다.

목차

ADR-001 — 백엔드 프레임워크 — Spring Boot 3.5
ADR-002 — 소셜 로그인 — 카카오 OAuth2 단일 Provider 우선
ADR-003 — JWT 토큰 전달 — HttpOnly Cookie
ADR-004 — 사주 계산 — 외부 API 위임 (Ablecity)
ADR-005 — LLM 연동 — OpenAI GPT-4o-mini
ADR-006 — 비동기 콜백 — Semi-Sync PendingRegistry
ADR-007 — 캐시 — Caffeine 인프로세스 → Redis 전환 계획
ADR-008 — 패키지 구조 — DDD Layered Hybrid
ADR-009 — 데이터베이스 — MySQL 8.0
ADR-010 — 회원 탈퇴 — Soft Delete + 30일 익명화
ADR-011 — 인프라 — AWS 단일 리전, 단일 인스턴스 (MVP)
요약 테이블

배경

사주 계산, 외부 API 연동, JWT 인증, 비동기 콜백 처리 등 복합 기능을 가진 RESTful API 서버가 필요하다. 2인 소규모 팀이 빠르게 MVP를 출시해야 하는 제약이 있다.

결정

Spring Boot 3.5.13 (Java 17)을 선택한다.

근거 (기술)

- Spring Security로 OAuth2·JWT·필터 체인을 선언적으로 구성할 수 있어 보안 표면을 줄인다.
- JPA + Flyway로 스키마 변경을 코드로 추적한다.
- Resilience4j(Circuit Breaker) + Spring Retry가 외부 API 장애 대응에 그대로 활용된다.
- 팀 숙련도가 가장 높아 초기 개발 속도가 빠르다.

서비스 관점

“우리 서비스의 핵심 가치는 사주 기반 소개팅’이지, 프레임워크 선택이 아니다. 가장 빠르게 안정적인 API를 만들 수 있는 선택이 최선이다.”

결과 / 트레이드오프

- 보일러플레이트가 많아 초반 설정 비용이 크다.
- JVM 구동 시간이 Node.js/Python 대비 느리지만, 상시 서버 환경에서 실질적 영향 없음.

고려한 대안

- NestJS — TypeScript 친화적이나 팀 숙련도 부족으로 제외
- FastAPI — LLM 호출은 외부 API에 위임하므로 Python 이점 없음

비교

평가 항목	Spring Boot	NestJS	FastAPI
생태계 성숙도	★★★★★	★★★★	★★★
보안 내장 기능	Spring Security (완비)	직접 구현	직접 구현
ORM/마이그레이션	JPA + Flyway	TypeORM	SQLAlchemy
팀 숙련도	높음	중간	낮음

향후 확장 가능성

WebSocket(STOMP) 서버는 Spring WebSocket 모듈로 동일 인스턴스에서 확장 가능 (2단계 채팅). 트래픽 급증 시 Auto Scaling Group으로 수평 확장.

배경

국내 사용자 대상 서비스에서 회원 가입 마찰을 최소화해야 한다. 사주 기반 소개팅 특성상 빠른 온보딩이 매칭 풀 확보에 직결된다.

결정

카카오 OAuth2 인가코드 방식을 1차 MVP의 유일한 로그인 수단으로 채택한다.

근거 (기술)

- 국내 20~35세 주요 타겟의 카카오 계정 보유율이 90% 이상.
- 카카오 로그인 버튼 하나로 회원가입·로그인을 통합해 온보딩 이탈률을 낮춘다.
- 카카오 사용자 정보(닉네임, 프로필 이미지)를 그대로 활용해 프로필 입력 단계를 생략한다.

서비스 관점

“‘소개팅 앱에서 회원가입 폼을 채우는 사람은 없다.’ 소셜 로그인이 없으면 MVP 매칭 풀 자체가 형성되지 않는다.”

결과 / 트레이드오프

- 카카오 API 장애 시 로그인 자체가 불가능 → 2단계에서 네이버·구글 추가로 단일 장애점 해소.
- user_social_accounts 테이블은 멀티 provider 구조로 설계 → 추가 비용 없이 확장 가능.

고려한 대안

- 이메일/비밀번호 자체 로그인 — 운영 부담(비밀번호 찾기, 이메일 인증)이 크고 전환율이 낮아 제외
- 애플 로그인 — iOS 배포 시 필수지만 MVP는 웹/안드로이드 우선이므로 2단계로 연기

향후 확장 가능성

Spring Security OAuth2 Client 멀티 provider 설정으로 네이버·구글을 동일 구조에서 추가. DB 스키마(provider 컬럼) 변경 없음.

배경

API 인증에 JWT를 사용한다. 클라이언트에서 토큰을 어떻게 보관하고 서버에 전달할지 결정해야 한다.

결정

AccessToken과 RefreshToken 모두 HttpOnly, Secure, SameSite=Strict Cookie로 전달한다.

근거 (기술)

- XSS로 토큰이 탈취되면 계정이 완전히 노출된다. HttpOnly Cookie는 JS가 읽을 수 없어 이 공격을 차단한다.
- RefreshToken 쿠키 경로를 /api/v1/auth/reissue로 제한해 불필요한 전송을 방지한다.
- Token Rotation 패턴(사용된 RT 즉시 폐기)으로 탈취된 RT의 재사용을 막는다.

서비스 관점

“사용자가 로그아웃하지 않고 재방문할 때 자동 갱신이 끊임없이 동작해야 재방문율이 유지된다. HttpOnly Cookie + Rotation이 이 경험을 만든다.”

결과 / 트레이드오프

- CORS 환경에서 credentials: include 설정 필요 — 프론트엔드와 협의 필수.
- 모바일 WebView에서 쿠키 동작 검증 필요.

고려한 대안

- LocalStorage + Authorization 헤더 — XSS로 토큰 탈취 가능하여 보안 위험

비교

방식	XSS 위험	CSRF 위험	구현 난이도
LocalStorage + 헤더	높음 (JS로 접근 가능)	낮음	단순
HttpOnly Cookie (채택)	없음 (JS 접근 불가)	낮음 (SameSite=Strict)	중간

향후 확장 가능성

네이티브 앱(iOS/Android) 전환 시 Authorization: Bearer 헤더 방식을 병행 지원하고, 토큰 저장소를 Secure Enclave / Keystore로 교체.

배경

사주 8자 계산(음력↔양력 변환, 천간지지 배치, 오행 비율 산출)은 수십 년간 검증된 동양 역학 알고리즘이 필요하다. 직접 구현 시 정확도 보장이 어렵다.

결정

사주 계산 전체를 Ablecity API에 위임한다. 자체 계산 엔진은 갖지 않는다.

근거 (기술)

- 음력 변환과 사주 배치는 옛지 케이스(절기 경계, 윤달 처리)가 많아 자체 구현 시 수개월이 소요된다.
- Ablecity는 국내 역학 SaaS로 이미 수백만 케이스를 검증한 엔진을 제공한다.
- '사주 계산 정확도 오차 0일(100케이스)'이라는 MVP KPI를 외부 API로 빠르게 달성한다.
- 오행 궁합 점수 계산의 원천 데이터도 Ablecity가 제공해 매칭 알고리즘의 신뢰성을 확보한다.

서비스 관점

“우리가 소개팅 앱을 만드는 것이지, 역학 계산기를 만드는 것이 아니다.” 핵심 가치는 사람 연결에 있다.”

결과 / 트레이드오프

- Ablecity API 장애 시 사주 등록·기운 조회가 전면 중단 → Circuit Breaker + Retry로 완화.
- 비용이 API 호출 수에 비례해 증가 → 캐싱으로 중복 호출 억제.

고려한 대안

- 직접 구현 — 정확도 보장 불가, 개발 기간 3개월 이상 추가 소요
- 오픈소스 라이브러리 — 국내 역학 기준 검증이 된 Java 라이브러리 없음

향후 확장 가능성

사용자 수 증가로 API 비용이 임계치를 넘으면, 생년월일→8자 결과를 DB에 캐싱해 반복 호출을 줄인다.

배경

Ablecity API는 수치(오행 비율, 점수)와 짧은 텍스트만 반환한다. 사용자에게 맥락 있는 운세 해설을 전달하려면 자연어 생성이 필요하다.

결정

OpenAI GPT-4o-mini를 LLM 엔진으로 사용한다.

근거 (기술)

- 일일 기운 분석·전통 운세 해설은 30~100단어 수준의 텍스트 생성으로 mini 모델로 충분하다.
- 비용이 중요한 초기 단계에서 GPT-4o 대비 약 1/5 수준의 비용이 경제적이다.
- 응답 속도가 빠르고 한국어 품질이 양호하다.

서비스 관점

“하루에 수천 명이 오늘의 기운을 조회하면 LLM 비용이 즉각 발생한다. 사주 정보를 무료로 제공하는 서비스 구조에서 비용 통제가 지속 가능성에 직결된다.”

결과 / 트레이드오프

- 할루시네이션 가능성 → 프롬프트에 Ablecity 수치를 앵커로 제공해 이탈 범위 제한.
- OpenAI API 장애 시 LLM 의존 기능(일일 기운 분석, 전통 운세)이 중단된다.

고려한 대안

- GPT-4o — 품질은 높지만 5배 비용, 초기 단계에서 과도
- Claude Haiku — 품질·비용 유사, 추후 A/B 테스트 후 교체 검토

비교

모델	품질	비용	응답속도	국어 품질
GPT-4o	★★★★★	높음	느림	우수
GPT-4o-mini (채택)	★★★★	낮음	빠름	양호
Claude Haiku	★★★★	낮음	빠름	양호

향후 확장 가능성

2단계에서 Claude API와 A/B 테스트해 국어 품질과 비용을 재평가. 프롬프트를 외부화하면 모델 교체 시 코드 수정 없음.

배경

Ablecity 신년운세·궁합 분석은 응답까지 수 초~수십 초 소요. 동기 HTTP 응답으로는 클라이언트 타임아웃과 DB 커넥션 고갈 두 문제가 동시에 발생한다.

결정

PendingRegistry(ConcurrentHashMap) + 120초 대기 + PROPAGATION.NOT_SUPPORTED 패턴을 채택한다.

근거 (기술)

- PROPAGATION.NOT_SUPPORTED로 메서드 진입 시 트랜잭션을 중단시켜 DB 커넥션을 반납한다.
- 클라이언트는 폴링 없이 한 번의 요청으로 결과를 받아 UX 단순성이 유지된다.
- 120초 타임아웃 시 PENDING 상태를 반환하고, 재요청 시 DB 캐시를 반환한다.

서비스 관점

“결과 화면에서 스피너가 돌다가 불러오기 버튼이 나타나는 것’보다 ‘잠깐 기다리면 바로 결과가 나오는 것’이 사용자 경험에 유리하다.”

결과 / 트레이드오프

- 단일 인스턴스 환경에서만 동작 — 인스턴스 2대 이상이면 콜백이 다른 인스턴스에 도달해 Future를 찾지 못함.
- 스케일아웃 시 Redis Pub/Sub으로 전환 필요.

고려한 대안

- 완전 비동기 폴링 — 클라이언트가 주기적으로 재요청해야 해 UX 복잡도 증가
- WebSocket Push — 실시간이나 구현 복잡도가 높고 MVP 단계에서 과도

비교

방식	클라이언트 경험	복잡도	커넥션 사용
동기 대기 (트랜잭션 열린 채)	단순	낮음	커넥션 고갈 위험
Semi-sync (채택)	대기 후 결과 수신	중간	커넥션 반납 후 대기
완전 비동기 (폴링)	폴링 필요	높음	없음

향후 확장 가능성

2단계 스케일아웃 시 Redis Pub/Sub으로 전환. PendingRegistry 인터페이스를 교체하는 것으로 비즈니스 로직 변경 없이 전환 가능.

배경

운세 결과(LLM 호출 결과)는 사용자·날짜 기준으로 하루 한 번만 생성하면 된다. 매 요청마다 OpenAI API를 호출하면 비용과 응답 시간이 증가한다.

결정

MVP에서는 Caffeine 인프로세스 캐시(24h TTL)를 사용한다. 스케일아웃 시 Redis로 전환한다.

근거 (기술)

- 단일 인스턴스 MVP에서 Caffeine은 외부 의존성 없이 마이크로초 수준 캐시를 제공한다.
- CaffeineCacheManager를 RedisCacheManager로 교체 시 비즈니스 로직 코드 변경이 없다.
- Redis 운영 비용과 복잡도를 트래픽이 검증될 때까지 지연한다.

서비스 관점

“MVP는 빠른 출시와 비용 통제가 우선이다. 사주 정보를 무료로 제공하는 구조에서 인프라 비용 최소화가 중요하다.”

결과 / 트레이드오프

- 인스턴스 2대 이상이면 캐시 불일치 발생 → 이 시점에 Redis로 전환.

고려한 대안

- 처음부터 Redis — 단일 인스턴스에서는 과도한 복잡도, ElastiCache 비용 즉시 발생
- Memcached — TTL 제어와 자료구조 다양성이 Redis 대비 부족 (Pub/Sub 활용 불가)

비교

기능	현재 구현	Redis 전환 시	전환 우선순위
운세 결과 캐시	Caffeine (24h TTL)	RedisCacheManager 교체 (코드 변경 없음)	1순위
Refresh Token	MySQL JPA	refresh: {userId} TTL 7일	1순위
Rate Limiting	미구현	sliding window counter	2순위
Pending Registry	ConcurrentHashMap	Redis Pub/Sub	3순위

향후 확장 가능성

인스턴스 2대 이상 전환 시 Redis로 교체. Refresh Token, Rate Limiting, Pub/Sub도 동일 Redis 인스턴스로 통합.

배경

기능이 늘어날수록 단순 계층형(Controller-Service-Repository) 구조는 도메인 간 의존성이 뒤엉킨다.

결정

DDD 기반 도메인 우선 패키지 구조를 채택한다 (api / application / domain / infra).

근거 (기술)

- 도메인 경계가 명확해 기능 추가 시 변경 범위가 한 도메인 안에 국한된다.
- domain/model/은 Spring-JPA 의존이 없어 단위 테스트가 빠르다.
- infra/ 교체(Caffeine→Redis, MySQL→PostgreSQL)가 비즈니스 로직에 영향을 주지 않는다.

서비스 관점

“2단계(매칭, 채팅)·3단계(커뮤니티)가 계획되어 있다. 도메인 분리 구조에서 신규 기능은 새 도메인 패키지를 추가하는 것으로 기존 코드를 건드리지 않는다.”

결과 / 트레이드오프

- 초반에 단순 계층형보다 파일이 많고 구조가 복잡하다.
- 팀원 온보딩 시 DDD 개념 교육이 필요하다.

고려한 대안

- 단순 계층형 — 초반에 빠르지만 도메인이 5개를 넘으면 순환 의존 발생
- 완전한 헥사고날 — 포트/어댑터 모두 인터페이스로 추상화하면 소규모 팀에 오버헤드

향후 확장 가능성

도메인 수가 10개를 넘거나 팀이 3명 이상으로 커지면 멀티 모듈(Gradle multi-project)로 분리해 경계를 강제한다.

배경

사용자 프로필, 사주 데이터, 운세 결과, 매칭 이력, 채팅 메시지 등 관계형 데이터를 영구 저장한다.

결정

MySQL 8.0 (AWS RDS)을 메인 데이터베이스로 사용한다.

근거 (기술)

- 관계형 데이터 모델이 사주(1:1)·매칭(N:M)·채팅(1:N) 구조에 자연스럽게 맞는다.
- AWS RDS MySQL은 자동 백업, 읽기 복제본, 스냅샷이 기본 제공된다.
- Flyway로 스키마 변경을 코드로 관리해 환경 간 일관성을 보장한다.
- 사주 8자를 JSON이 아닌 개별 컬럼으로 저장해 오행 기반 궁합 점수 쿼리를 지원한다.

서비스 관점

“매칭·채팅 데이터는 ACID 보장이 필수다. NoSQL은 상호 좋아요→매칭 생성 트랜잭션에서 데이터 정합성 보장이 복잡해진다.”

결과 / 트레이드오프

- 스키마 변경 시 Flyway 마이그레이션 파일 작성이 필수다.
- 채팅 메시지가 급증하면 별도 스토리지 분리를 검토한다.

고려한 대안

- PostgreSQL — JSON 지원이 더 강력하지만 팀 운영 경험이 MySQL에 집중
- MongoDB — 채팅 메시지 저장에 유리하지만 매칭 트랜잭션 보장이 복잡

향후 확장 가능성

DAU 5,000+ 단계에서 RDS Read Replica를 추가해 조회 부하를 분산한다. 채팅 메시지는 DynamoDB 분리를 장기적으로 검토한다.

배경

GDPR·개인정보보호법은 탈퇴 시 개인정보 삭제 의무를 부과하지만, 매칭·채팅 이력은 상대방 기록이기도 해 일정 기간 보존이 필요하다. 탈퇴 직후 완전 삭제는 오삭제 복구가 불가능하다.

결정

Soft Delete(deletedAt 컬럼) + 30일 유예 + 매일 02:00 KST 익명화 스케줄러를 채택한다.

근거 (기술)

- 탈퇴 후 30일 이내 복구 기능으로 오탈퇴 사용자를 구제한다.
- 30일 후 익명화로 개인정보 삭제 의무를 이행한다.
- 매칭·채팅 이력 레코드는 식별자(userId FK)가 남아 상대방 대화 이력을 보존한다.

서비스 관점

“‘실수로 탈퇴했어요’ CS가 반드시 발생한다. 30일 유예는 이 비용을 CS 개입 없이 자체 해결하게 한다. 동시에 30일 후 자동으로 GDPR 의무를 이행해 법적 위험을 줄인다.”

결과 / 트레이드오프

- 탈퇴 신청 후 30일간 PENDING_DELETION 상태로 API 접근이 제한된다.
- 매일 02:00 KST 스케줄러가 email-nickname을 익명화한다.

고려한 대안

- 즉시 완전 삭제 — 오삭제 복구 불가, 상대방 채팅 이력 파괴
- 60일 유예 — 개인정보 처리 기간이 길어져 법적 리스크 증가

향후 확장 가능성

탈퇴 신청 시 카카오 메시지로 ‘30일 이내 복구 안내’를 발송하면 복구율이 높아진다.

배경

2인 팀 MVP 단계에서 인프라 복잡도와 비용을 최소화해야 한다.

결정

AWS 서울 리전(ap-northeast-2), EC2 단일 인스턴스, RDS MySQL, S3, Route53 + ACM으로 구성한다.

근거 (기술)

- 국내 서비스로 서울 리전이 레이턴시와 데이터 주권 모두 최적이다.
- RDS는 자동 백업·페일오버를 관리형으로 제공해 DB 운영 부담을 없앤다.
- GitHub Actions + Docker로 코드 푸시 → 자동 배포를 초기부터 확보한다.

서비스 관점

“초기 비용: EC2(t3.small) ~\$20/월 + RDS(db.t3.micro) ~\$15/월 + 기타 ~\$10 = 월 \$50 미만. 사용자와 수익이 생긴 후 인프라를 고도화한다.”

결과 / 트레이드오프

- 단일 인스턴스로 PendingRegistry·Caffeine 캐시가 인스턴스 내에 존재해 스케일아웃 제약.
- 인스턴스 2대 이상 전환 시 Redis로 상태를 외부화해야 한다.

고려한 대안

- 처음부터 MSA + 쿠버네티스 — 2인 팀에 운영 부담이 너무 크고 비용이 10배 이상
- Vercel + PlanetScale — 서버리스 환경은 PendingRegistry 패턴 구현 불가

비교

트리거	전환 내용
DAU 1,000+	Redis ElastiCache 추가, Caffeine → Redis 캐시 전환
DAU 5,000+	EC2 Auto Scaling Group + ALB, RDS Read Replica
채팅 기능(2단계)	WebSocket 전용 인스턴스 또는 MSA 분리
글로벌 확장	CloudFront CDN, 멀티 AZ RDS

향후 확장 가능성

DAU 1,000+ 시 Redis 추가. DAU 5,000+ 시 Auto Scaling Group + ALB + RDS Read Replica. 채팅 기능(2단계) 시 WebSocket 전용 인스턴스 분리 검토.

요약 — 결정 목록

#	결정	선택	주요 제외 대안
ADR-001	백엔드 프레임워크	Spring Boot 3.5	NestJS, FastAPI
ADR-002	소셜 로그인	카카오 OAuth2	자체 이메일, 애플
ADR-003	토큰 전달	HttpOnly Cookie	LocalStorage
ADR-004	사주 계산	Ablecity API 위임	자체 구현
ADR-005	LLM	GPT-4o-mini	GPT-4o, Claude
ADR-006	비동기 콜백	Semi-sync PendingRegistry	폴링, WebSocket Push
ADR-007	캐시	Caffeine → Redis(계획)	처음부터 Redis, Memcached
ADR-008	패키지 구조	DDD Layered Hybrid	단순 계층형, 완전 헥사고날
ADR-009	데이터베이스	MySQL 8.0	PostgreSQL, MongoDB
ADR-010	탈퇴 처리	Soft Delete + 30일 익명화	즉시 삭제
ADR-011	인프라	AWS 단일 인스턴스 MVP	MSA, 쿠버네티스